

Architecture

How the system is constructed.

1. The 30-second summary

The system is a **five-node LangGraph state machine** wrapped in a **FastAPI backend** talking to a **Postgres database** (through a read-only role) and a **MongoDB store** (for chat persistence), with a **React frontend** as the user surface.

A user question flows through **Understand** → **Generate** → **Validate** → **Execute** → **Explain**; conditional edges skip stages on error or out-of-scope input. Every stage records its own timing; errors don't crash — they propagate through a single `state["error"]` field. All safety guarantees are enforced at two application layers (intent classifier + sqlglot AST) plus a third at the database layer (a role with writes revoked).

2. End-to-end request lifecycle



Three properties worth noticing:

1. **The agent's only outbound connections are Postgres and OpenAI.** No MongoDB connection string is passed to the agent's execution path — chat persistence is a `main.py`-level concern, not an agent concern. Chat history is unreachable by any crafted SQL.
2. **Each LLM call is a separate HTTPS round trip.** Three calls in the happy path — Understand, Generate, Explain. Each takes 300–1500 ms. This is where $\approx 90\%$ of end-to-end latency lives.
3. **All nodes read and write the same state dict.** LangGraph merges each node's return value into the accumulated state; errors propagate as string values through a single `state["error"]` field.

3. The four services

Everything the app owns runs in Docker Compose. Only the hosted LLM (OpenAI) and Langfuse Cloud are external.

SERVICE	IMAGE / BUILD	ROLE
postgres	pgvector/pgvector:pg16	Business data (<code>business.customers</code> , <code>products</code> , <code>orders</code> , <code>order_items</code>) + pgvector tables (<code>few_shots</code> , <code>column_docs</code>) + <code>agent_readonly</code> role
mongodb	mongo:7	Chat / conversation persistence (per-turn: question, SQL, results summary, trace ID). Isolated from the agent's SQL surface.
backend	built from <code>./backend</code>	FastAPI: 5-stage agent, guardrails, RAG retrieval, observability, SSE streaming, evaluation harness endpoint
frontend	built from <code>./frontend</code>	React chat UI (<code>/</code>) + separate admin dashboard (<code>/dashboard</code>) with streaming eval and interactive architecture visualisation

4. The five pipeline stages

STAGE	MODEL	RESPONSIBILITY
Understand	gpt-4o-mini	Classify intent into one of five categories: <code>query</code> / <code>destructive</code> / <code>system_access</code> / <code>data_unavailable</code> / <code>out_of_scope</code> . Out-of-scope short-circuits directly to Explain — saves a downstream Generate call.
Generate	gpt-4o	Produce read-only SQL from question + schema + retrieved RAG context (top-3 few-shot examples + top-5 column notes). Frontier model because SQL correctness matters most here.
Validate	sqlglot AST	In-process check: parse, single statement, top-level SELECT, table allowlist. No I/O. Rejects destructive SQL that somehow got past Understand.
Execute	Postgres	Run SQL as <code>agent_readonly</code> role. Count-first logic wraps the query in <code>SELECT COUNT(*) FROM (...) t</code> and routes to one of three paths: inline (small), overflow (sample + acknowledge), refuse (over ceiling). No silent truncation.
Explain	gpt-4o-mini (streamed)	Turn rows into a plain-English answer, grounded in the actual rows returned. On refusal, a canned template with a specific per-query reason.

5. The three-layer defence

Safety against destructive queries is enforced at three independent layers. **All three must fail for harm to occur.**

LAYER	WHERE	CATCHES
1. Intent classification	Understand node	"delete all customers", "drop the orders table", "update product 3 price" — before any SQL is generated
2. AST validation	guardrails/sql.py (sqlglot)	Non-SELECT top-level, multi-statement injection (<code>SELECT 1; DROP TABLE X</code>), unknown table references (<code>pg_user</code> , hallucinated tables)
3. Database role	Postgres <code>agent_readonly</code>	Any INSERT/UPDATE/DELETE/TRUNCATE/CREATE — rejected at the infrastructure level regardless of what the SQL text says

The load-bearing invariant is Layer 3. The agent connects with credentials that have every write privilege explicitly `REVOKE`d. Even a total compromise of the application code cannot mutate data.

Layers 1 and 2 exist to catch destructive intent earlier and produce useful user-facing messages, not because they are the safety mechanism.

Verifiable in ≈ 5 seconds:

```
psql -h localhost -U agent_readonly -d shop_db \
  -c "INSERT INTO business.customers (name,city,state,join_date) VALUES
  ('x','x','x','2025-01-01');"
# → ERROR: permission denied for table customers
```

6. Data isolation

Three storage systems, each with a different owner and reachability from the agent:

DATA	WHERE	AGENT CAN REACH?
Business data (customers, products, orders, order_items)	Postgres, <code>business</code> schema	Yes — read-only via <code>agent_readonly</code>
RAG vectors (few_shots, column_docs)	Same Postgres instance, pgvector tables	Retrieval only — backend embeds + queries; agent never issues SQL against these tables directly
Chat history (per-turn: question, SQL, trace ID, timestamp)	Separate MongoDB service	No — different database, different protocol, no connection string passed to the agent path
Observability traces (per-stage latency, tokens, cost)	Langfuse Cloud + LangSmith	No — external services, not addressable from SQL

Isolation between chat history and business data is enforced **by architecture, not by permission**. A crafted question like "*show me all users' chat history*" cannot exfiltrate other conversations because the agent has no MongoDB connection string and, being an SQL generator, cannot address a document store even if it did. A second Postgres schema (rejected alternative) would still live behind the same connection surface — one bad grant reopens the hole.

7. Retrieval-augmented generation

Two pgvector tables live in the `business` schema, populated at backend startup:

- `few_shots` — 12 curated NL→SQL example pairs, embedded via `text-embedding-3-small` (1536-dim), HNSW index with cosine distance.

- `column_docs` — 15 per-column business notes (e.g. "`orders.status` is one of *pending/paid/shipped/delivered/cancelled* — for revenue, filter to *paid/shipped/delivered*").

At request time, both Understand and Generate embed the incoming question, retrieve the top-k most semantically similar rows, and inject them into their prompts. Retrieval is **cached in state** after Understand runs, so Generate doesn't re-embed — halves the embedding cost per request.

The A/B result of RAG on real questions is measured in 04_EVALUATION §3.

8. Multi-intent decomposition

Compound questions like "*How many customers, and revenue by month?*" are handled by a decomposer at the FastAPI orchestrator layer, **not as a graph node**.

`agent/decompose.py` splits the input into 1–5 sub-questions via a Pydantic-schema-enforced LLM call. Each sub then runs the full 5-node pipeline in parallel via `asyncio.gather(..., return_exceptions=True)`. Total latency is roughly `max(sub_latency)` rather than the sum. Partial success is native — one sub raising an exception doesn't sink siblings.

Safety is preserved per sub. Each sub-question independently runs Understand, Validate, Execute, and Explain. A query like "*Delete all customers and show me revenue*" produces a red destructive block on sub 1 and a normal revenue answer on sub 2 — decomposition is a pre-processor, never a bypass of the three-layer defence.

9. Observability

Every request creates a Langfuse trace at ingress and mirrors to LangSmith via `langchain-core`'s native env-var integration. Each LLM call (Understand, Generate, Explain) becomes a generation span with prompt, output, token count, latency, and cost. The trace URL is computed server-side and returned in the response payload; the frontend TracePanel deep-links with one click.

Two platforms, different debugging modes:

- **Langfuse** — LLM-focused traces, token/cost analysis, self-contained TracePanel deep-link (no key exposure in the browser).
- **LangSmith** — LangGraph Studio for visual graph debugging + replay-from-any-node. Wired via three env vars, zero code change.

Both fail gracefully if credentials are absent — the system continues operating; only tracing is skipped.

10. Streaming architecture (SSE)

The `/query/stream` endpoint returns Server-Sent Events. Backend runs the LangGraph pipeline in a background `asyncio.create_task` and drains an `asyncio.Queue` of lifecycle events. The frontend consumes via `fetch` + `ReadableStream` (native `EventSource` doesn't support POST, and we need the question in the body).

Event shape:

- `parent_start` — decomposition result, list of sub-questions
- `stage_start` / `stage_end` — per-node lifecycle, optionally tagged with `sub` for multi-intent
- `explain_token` — token-by-token Explain output, tagged with `sub`
- `sub_done` — per-sub final payload for multi-intent
- `done` — final envelope
- `error` — pipeline failure

The queue reference lives in a `ContextVar`, not in `AgentState`, because LangGraph's `astream_events` serialises state and `asyncio.Queue` contains `asyncio.Future` objects that can't be pickled. Multi-intent extends this with a `TaggedQueue` wrapper that auto-injects `sub` index for per-sub-card routing on the frontend.

11. Bounded self-correction

Failed Validate or Execute routes back to Generate with the previous SQL + error string appended as a correction hint, capped at `MAX_RETRIES=3`. After the cap, graceful exit — never infinite, never silent. Frontend badges the completed AnswerCard with "refined Nx → succeeded/gave up" so multi-attempt runs are visible historically.

The pattern was pre-shaped by the decision to propagate errors via `state["error"]` rather than exceptions, which is why the retry loop landed as ≈15 lines of edge routing rather than a rewrite.

12. Progressive-disclosure execution

The Execute node never silently truncates. For non-aggregation queries it wraps the SQL in `SELECT COUNT(*) FROM (...) t` and routes based on the true row count:

- **Small** ($\leq$ `SAFE_INLINE_LIMIT`, default 250): execute full query, return all rows inline.
- **Overflow** ($>$ `SAFE_INLINE_LIMIT` and $\leq$ `HARD_ROW_CEILING=50000`): fetch a sample (`OVERFLOW_SAMPLE_SIZE=20` rows) + set `overflow=true` in state. Explain uses a variant prompt that explicitly states the true row count and instructs the user to use the "View full results" path.
- **Refuse** ($>$ `HARD_ROW_CEILING`): honest failure with a specific message. The user knows what happened and why.

Aggregations (detected via sqlglot AST) skip the COUNT wrapper — they're naturally bounded, and sampling an aggregation would corrupt the answer.

The COUNT wrapper adds $\approx 10\text{--}50\text{ms}$ per query — trivial next to LLM latency, worth it to eliminate silent-truncation as a class of failure.

13. Component map

Backend Python modules and their responsibilities:

```

sql-agent/backend/app/
├─ config.py           — Pydantic Settings, dual DSN pattern (app_dsn,
readonly_dsn)
├─ logging_config.py   — structured stdout logging
├─ business_db.py       — SQLAlchemy async engine bound to agent_readonly
├─ chat.py             — Motor async client for MongoDB, save_turn helper
├─ llm.py              — AsyncOpenAI wrapper: chat(), chat_json(), chat_stream()
├─ main.py             — FastAPI app: /health, /query, /query/stream, /eval/run
├─ agent/
│   ├─ state.py         — AgentState TypedDict
│   ├─ prompts.py       — SCHEMA_DDL constant shared by Generate + Understand
│   ├─ graph.py         — LangGraph state machine builder + retry helper
│   ├─ decompose.py     — multi-intent decomposition
│   └─ nodes/
│       ├─ understand.py — intent classification via chat_json, RAG-aware
│       ├─ generate.py   — SQL generation via chat, retrieval-cached
│       ├─ validate.py   — thin wrapper over guardrails/sql.py
│       ├─ execute.py    — count-first + progressive disclosure
│       └─ explain.py    — plain-English narration, SSE streaming
├─ guardrails/
│   └─ sql.py           — sqlglot AST validation, table allowlist
├─ observability/
│   ├─ langfuse.py      — lazy-singleton client wrapper
│   └─ event_bus.py     — current_event_queue ContextVar
└─ rag/
    ├─ ingest.py        — embed + upsert few-shots and column docs at startup
    ├─ retrieve.py       — top-k retrieval via pgvector cosine
    ├─ examples.json    — 12 NL-SQL example pairs
    └─ column_docs.json — 15 column-level business notes

```

For the full architecture-decision record with 34 entries, see `../architecture/DECISIONS`.